



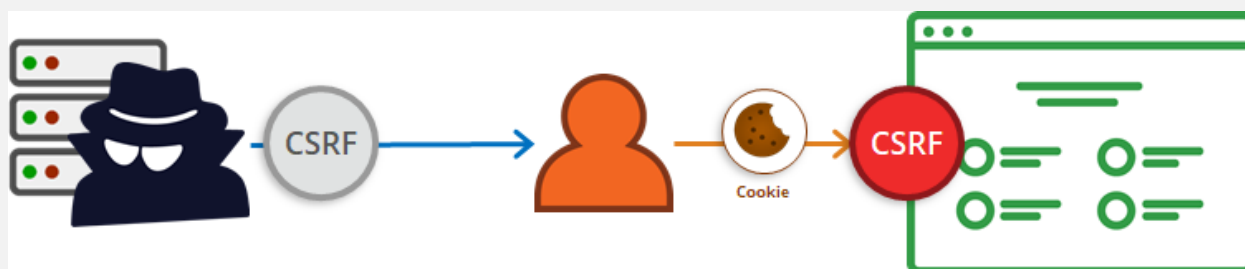
АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Друзья, всем доброго времени суток!

В этом уроке мы поговорим про CSRF. Подделка межсайтовых запросов (CSRF) — это Client Side атака (атака на стороне клиента), которая заставляет конечного пользователя выполнять нежелательные действия в веб-приложении, в котором он в данный момент аутентифицирован. С помощью социальной инженерии (например, отправкой ссылки по электронной почте или в чате) злоумышленник может обмануть пользователей веб-приложения, заставив их выполнить действия по выбору злоумышленника. Если жертва — обычный пользователь, успешная CSRF атака может заставить пользователя выполнить действия, такие как перевод средств, изменение адреса электронной почты и т. д. Если жертвой является администратор веб-приложения, CSRF может поставить под угрозу все веб-приложение.

CSRF — это атака, которая обманом заставляет жертву отправить вредоносный запрос. Для большинства сайтов запросы браузера автоматически включают любые учетные данные, связанные с сайтом, такие как сеансовый cookie пользователя, IP-адрес и т. д. Поэтому, если пользователь в данный момент аутентифицирован на сайте, сайт не сможет отличить поддельный запрос, отправленный жертвой, от легитимного запроса, отправленного жертвой.



CSRF-атаки нацелены на изменение состояния на сервере, например, изменение адреса электронной почты или пароля жертвы или покупка чего-либо. В данном случае извлечение пользовательских данных не приносит пользы злоумышленнику, поскольку злоумышленник не получает ответ, а жертва получает его. Таким образом, CSRF-атаки нацелены на запросы, изменяющие состояние.

Атаки CSRF также известны под рядом других названий: XSRF, «Sea Surf», Session Riding, Cross-Site Reference Forgery и Hostile Linking.

Как работает CSRF-атака?

Чтобы выполнить атаку, мы должны сначала понять, как сгенерировать действительный вредоносный запрос для нашей жертвы. Давайте рассмотрим следующий пример: Катя хочет перевести 100 рублей Мише с помощью веб-приложения vuln.ru, которое уязвимо для CSRF. Мария, злоумышленник, хочет обманом заставить Катю отправить деньги Марии. Атака будет включать следующие шаги:

- Создание URL-адреса или скрипта эксплойта;
- Обманом заставить Катю выполнить действие с помощью социальной инженерии.

Если бы приложение было разработано в первую очередь для использования GET-запросов для передачи параметров и выполнения действий, операция перевода денег могла бы быть сведена к следующему запросу:

```
GET http://vuln.ru/transfer.do?acct=Mike&amount=100 HTTP/1.1
```

Теперь Мария решает использовать уязвимость этого веб-приложения, используя Катю в качестве жертвы. Сначала Мария создает следующий URL-адрес, который переведет 100 000 рублей со счета Кати на счет Марии. Мария берет исходный URL-адрес и заменяет имя получателя на себя, значительно увеличивая сумму перевода:

```
http://vuln.ru/transfer.do?acct=Maria&amount=100000
```

Далее с помощью техник социальной инженерии Мария заставляет перейти Катю по вредоносной ссылке.

Вредоносный URL-адрес можно замаскировать под обычную ссылку, побуждая жертву щелкнуть по ней:

```
<a href="http://vuln.ru/transfer.do?acct=MARIA&amount=100000">Забери приз!</a>
```

Или как изображение с размером 0 на 0:

```

```

Если бы этот тег изображения был включен в электронное письмо, Катя бы ничего не заметила. При этом браузер отправит запрос на vuln.ru без какого-либо визуального указания на то, что перевод был выполнен.

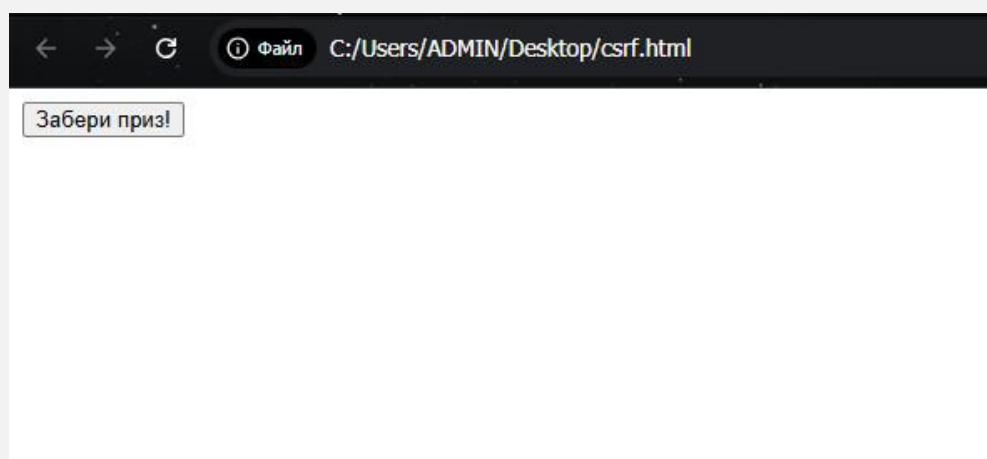
Единственное различие между атаками GET и POST заключается в том, как запрос выполняет жертва. Предположим, что сайт теперь использует POST запросы, а уязвимый запрос выглядит следующим образом:

```
POST http://vuln.ru/transfer.do HTTP/1.1
acct=Mike&amount=100
```

Такой запрос не может быть доставлен с использованием стандартных тегов А или IMG, но может быть доставлен с использованием HTML-форм:

```
<form action="http://vuln.ru/transfer.do" method="POST">
<input type="hidden" name="acct" value="MARIA"/>
<input type="hidden" name="amount" value="100000"/>
<input type="submit" value="Забери приз!"/>
</form>
```

Для жертвы эта вредоносная веб-страница будет выглядеть следующим образом:



Эта форма потребует от пользователя нажать кнопку “Забери приз!”, но это также можно выполнить автоматически (без нажатия) с помощью JavaScript:

```
<body onload="document.forms[0].submit()">  
<form...
```

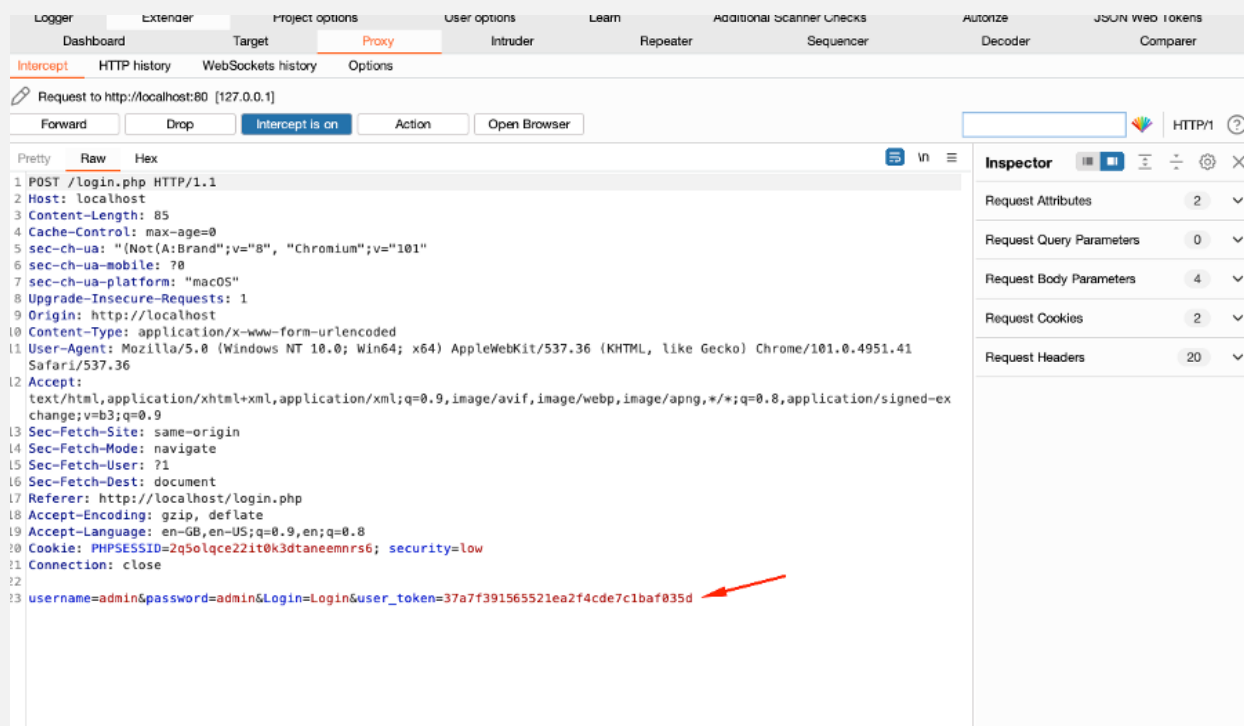
Какие существуют методы для предотвращения CSRF-атак?

CSRF-атакам сравнительно просто противостоять. Для этого используют две меры безопасности: реализация CSRF-токенов и использование атрибута SameSite в файлах cookie.

Для защиты от CSRF-атак распространенной мерой безопасности является внедрение CSRF-токенов.

CSRF-токены — это уникальные значения, которые случайным образом генерируются на стороне сервера и отправляются клиенту. Поскольку эти значения уникальны для каждого запроса, они повышают безопасность приложения, затрудняя (если не исключая) возможность их угадывания злоумышленником и таким образом предотвращая использование уязвимости CSRF.

Некоторые фреймворки, такие как Django (Python) и Laravel (PHP) используют промежуточное программное обеспечение для защиты от CSRF-атак, которое активировано по умолчанию.



SameSite — это атрибут, который приложение может вставлять в файлы cookie. Действительно, когда этот атрибут помещается в файлы cookie, они не будут отправляться на сервер, если запрос не исходит из домена приложения. Приложение, получающее запрос, не сможет связать запрос с каким-либо пользователем и, следовательно, не будет его обрабатывать. Теоретически это обеспечивает полную защиту от CSRF-атак.

Однако атрибут SameSite не учитывает поддомены. Таким образом, пользователь, контролирующий поддомен, может скомпрометировать приложение с помощью атаки CSRF. Например, можно будет атаковать веб-приложение “vuln.ru” с поддомена “evil.vuln.ru”.

Давайте подробнее рассмотрим, как работает атрибут SameSite:

Приложение инструктирует браузер пользователя о необходимости создания cookie через заголовок “Set-Cookie”. Например, если мы имеем дело с приложением PHP/Laravel, весьма вероятно, что у нас будет заголовок со значением, похожим на это:

```
Set-Cookie: PHPSESSID=RcUSA7T4T9kE
```

Заголовок Set-Cookie позволяет использовать несколько флагов для повышения безопасности файлов cookie пользователя: SameSite, Expires, Domain...

SameSite можно использовать с разными значениями: Strict и Lax. Если атрибут установлен на “Strict” (SameSite=Strict), это означает, что браузер пользователя никогда не будет отправлять файлы cookie с межсайтовыми запросами. Файл cookie отправляется только с запросами, исходящими из того же домена и поддоменов.

```
Set-Cookie: PHPSESSID= RcUSA7T4T9kE; Max-Age=43200; Secure; HttpOnly;  
SameSite=Strict
```

Если установлен атрибут “Lax” (SameSite=Lax), файлы cookie будут отправляться при межсайтовых GET-запросах, но только если запрос является результатом навигации пользователя по верхнему уровню (top-level navigation). Например, когда пользователь переходит по ссылке. Top-level navigation - это тип навигации, при котором пользователь переходит между основными разделами или страницами сайта с помощью прямых

ссылок или кнопок, обычно расположенных в видимой и доступной части интерфейса, такой как панель навигации или меню. Когда пользователь нажимает на ссылку на главной странице сайта, чтобы перейти к разделу “О нас” или “Контакты”, это считается навигацией по верхнему уровню.

```
Set-Cookie: PHPSESSID=VxTGA2T1T0kE; Max-Age=43200; Secure; HttpOnly; SameSite=Lax
```

Если cookie установлен с атрибутом “None” (SameSite=None), это полностью отключает ограничения SameSite независимо от браузера (см. Как современные браузеры противостоят CSRF-атакам?). В результате браузеры будут отправлять файлы cookie во всех запросах к сайту, даже в тех случаях, когда эти запросы были вызваны совершенно несвязанными сторонними сайтами.

Явное определение атрибутов SameSite повышает безопасность от атак CSRF, поскольку не позволяет браузеру отправлять межсайтовые запросы, включающие сеансовые файлы cookie.

Как современные браузеры противостоят CSRF-атакам?

Первые атаки CSRF были зафиксированы еще в 2000 году. Тогда мер защит, по большому счету, еще не было. С тех пор ситуация изменилась.

Последние изменения в начале 2020 года были применены основными браузерами, такими как Mozilla Firefox и Google Chrome, заключающиеся в установке атрибута по умолчанию “SameSite=Lax” каждый раз, когда он не установлен веб-приложением. Это значительно сократило количество атак CSRF, и это одна из причин, по которой он больше не входит в десятку OWASP с 2017 года.

Подводя итог, CSRF (подделка межсайтовых запросов) — это тип атаки на пользователей веб-приложений, и его основная характеристика — обман аутентифицированного пользователя с целью выполнения нежелательного действия. Успешная атака CSRF зависит от слабого механизма аутентификации, который полагается исключительно на файлы cookie для обработки сеанса пользователя. Современные браузеры теперь применяют более строгие политики в отношении файлов cookie, в то время как некоторые фреймворки, такие как Django (Python) и Laravel (PHP), содержат

анти-CSRF механизмы. Против CSRF можно реализовать два основных способа защиты. Первый основан на атрибутах файлов cookie, а второй основан на реализации токенов CSRF в веб-приложении. Для повышения безопасности часто применяются оба метода.